RAG-Based Customer Support Assistant (LangGraph + HITL)

(Technical Document)

Name: Sakshi Suresh Dhurde

College: P R Pote College Of Engineering And Management, Amravati.

Course: Internship Project

Date: April 2026

Id: IN226096402

1. Introduction

◈ What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is a hybrid AI architecture that combines:

- 🔎 **Information Retrieval (IR)** → Fetch relevant data from external knowledge sources
- 🤘 **Large Language Model (LLM)** → Generate natural language responses

Instead of relying only on pre-trained knowledge, RAG systems dynamically retrieve relevant context from documents (PDFs, databases, etc.) and use it to generate accurate responses.

◈ Why RAG is Needed

Traditional LLMs suffer from:

- Outdated knowledge
- Hallucinations (incorrect answers)
- Lack of domain-specific understanding
- No access to private documents

RAG solves this by:

✓ Grounding responses in real data
✓ Improving factual accuracy

✓ Allowing dynamic knowledge updates
✓ Reducing hallucinations

◈ Use Case Overview

This project implements a **Customer Support Assistant** that:

- Reads company policy documents (PDFs)
- Retrieves relevant sections using embeddings
- Answers user queries contextually
- Escalates uncertain cases to humans (HITL)
- Uses LangGraph for structured workflow control

---

2. System Architecture Explanation

◈ High-Level Architecture

The system consists of the following layers:

1. User Interface Layer

- Streamlit-based chat interface
- Accepts user queries
- Displays responses

2. Document Ingestion Layer

- Loads PDF files
- Extracts raw text
- Performs chunking

3. Embedding Layer

- Converts text into vector embeddings
- Uses SentenceTransformers model

4. Vector Database Layer

- Stores embeddings in ChromaDB
- Enables semantic search

5. Retrieval Layer

- Retrieves most relevant chunks
- Uses similarity search

6. LLM Layer

- Generates final response using retrieved context

7. Workflow Orchestration Layer (LangGraph)

- Controls flow between modules
- Handles routing decisions

8. HITL Layer

- Escalates low-confidence queries to humans

◈ Component Interaction Flow

User Query → UI → LangGraph → Retriever → Vector DB → Context → LLM → Answer → UI

---

3. Design Decisions

◈ Chunk Size Choice

- Chunk size: 200–500 characters
- Overlap: 50–100 characters

Reason:

- Small chunks improve precision

- Overlap prevents loss of context
- Avoids mixing multiple policies

◈ Embedding Strategy

- Model: all-MiniLM-L6-v2
- Converts text into dense vector representation

Reason:

- Lightweight and fast
- Good semantic understanding
- Suitable for real-time applications

◈ Retrieval Approach

- Top-k similarity search (k=3–5)
- Cosine similarity in vector space

Reason:

- Balances accuracy and performance
- Reduces irrelevant context injection

◈ Prompt Design Logic

Prompt includes:

- Strict instruction constraints
- Retrieved context
- User query

Rules:

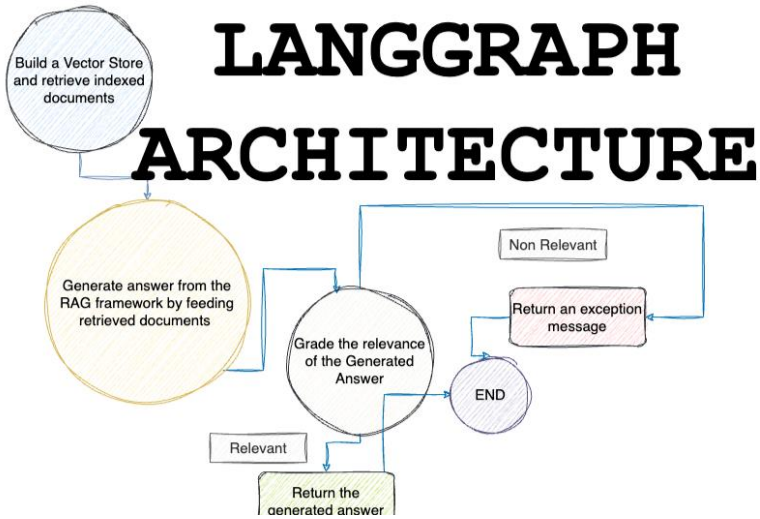
- Use only provided context
- Do not hallucinate
- Provide concise answers (1–3 lines)

---

4. Workflow Explanation (LangGraph)

◈ LangGraph Usage

LangGraph is used to define a **state-based workflow system** where each node represents a processing step.



◈ Nodes in System

1. Input Node → Accepts user query
2. Intent Node → Identifies query category
3. Retrieval Node → Fetches relevant chunks
4. LLM Node → Generates answer
5. HITL Node → Escalates uncertain cases
6. Output Node → Returns response

◈ State Transitions

Shared state object:

```
{
  "query": "",
  "intent": "",
  "context": [],
  "answer": "",
  "confidence": 0.0,
  "escalate": false
}
```

Flow:

Input → Intent → Retrieval → LLM → Confidence Check → Output / HITL

---

5. Conditional Logic

◈ Intent Detection

Classifies queries into:

- Refund-related
- Shipping-related
- Support-related
- Cancellation-related

◈ Routing Decisions

Based on:

- Intent type
- Retrieval score
- Confidence level

◈ Decision Rules

- High confidence → Direct answer
- Medium confidence → LLM refinement
- Low confidence → HITL escalation

---

6. HITL Implementation

◈ Role of Human Intervention

Human agents handle:

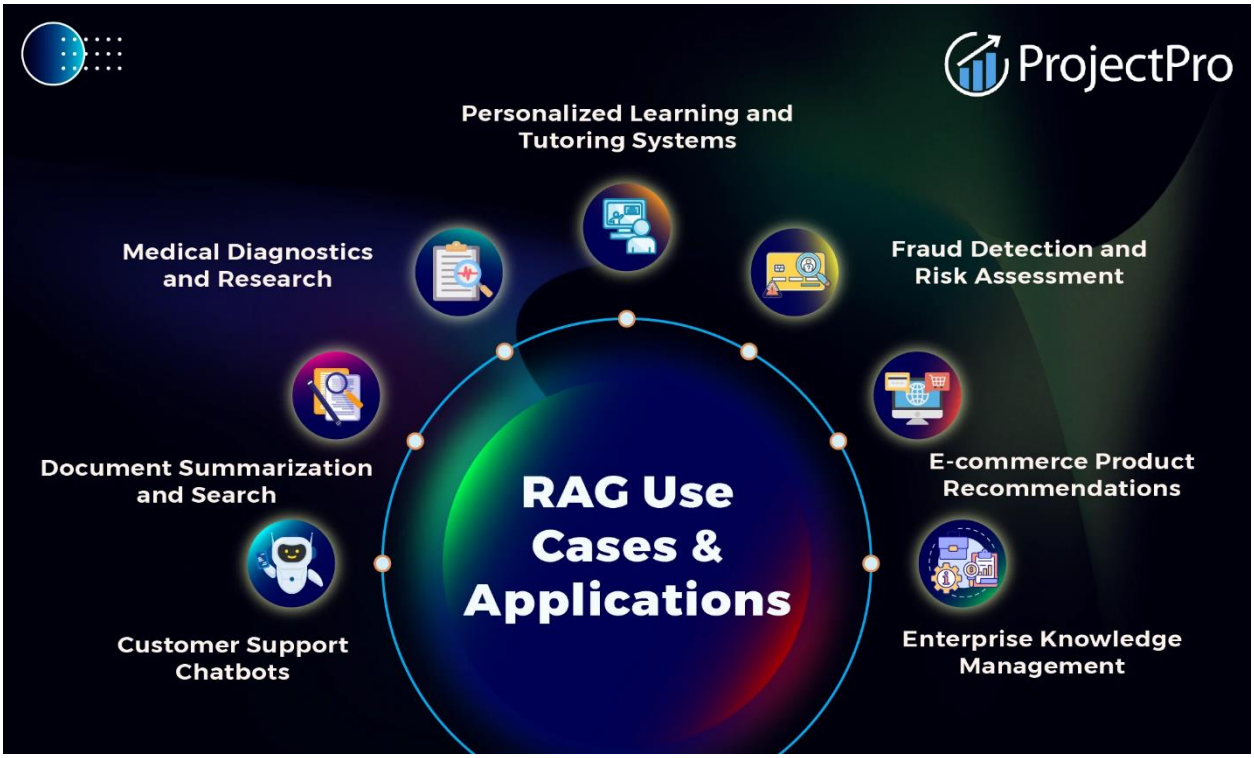
- Ambiguous queries
- Missing context cases
- Critical customer issues

◈ Workflow

1. System detects low confidence
2. Query is sent to human dashboard
3. Human provides response
4. Response returned to user
5. Stored for future improvement

◈ Benefits

✓ Improves accuracy
✓ Handles edge cases
✓ Reduces hallucinations



◈ Limitations

✕ Slower response time
✕ Requires human availability

---

7. Challenges & Trade-offs

◈ Retrieval Accuracy vs Speed

- Higher k improves accuracy
- Increases latency

◈ Chunk Size vs Context Quality

- Small chunks → precise but fragmented
- Large chunks → better context but noisy

◈ Cost vs Performance

- Using LLM increases cost
- Local models reduce cost but lower quality

---

8. Testing Strategy

◈ Testing Approach

- Unit testing (modules)
- Integration testing (pipeline)
- Query-based validation

◈ Sample Test Queries

- What is refund policy?
- How can I contact support?
- What is delivery time?
-

◈ Validation Metrics

- Accuracy of answers
- Retrieval relevance
- Response time
- HITL activation rate

---

9. Future Enhancements

◈ Multi-Document Support

- Support multiple PDFs
- Document-aware retrieval

◈ Feedback Loop

- User feedback stored
- Used for model improvement

◈ Memory Integration

- Remembers user queries
- Improves personalization

◈ Deployment Improvements

- FastAPI backend
- Docker containerization
- Cloud deployment (AWS/GCP)